

Resumo do dataset PrimeVul

Ding, Y. et al. Vulnerability Detection with Code Language Models: How Far Are We? ICSE 2025. Dados e código: github.com/DLVulDet/PrimeVul

235.768	funções C/C++
6.968	vulneráveis ($\approx 3\%$)
228.800	benignas
755	projetos
6.827	commits
140	tipos de falha (CWEs)
5.480	pares vulnerável $\times$ corrigida

1. O que é

O PrimeVul é um conjunto de dados para treinar e avaliar detectores de vulnerabilidade baseados em modelos de linguagem de código. Cada amostra é uma função C/C++ de um projeto de código aberto real, com rótulo binário: 1 = vulnerável, 0 = não vulnerável.

Além das funções avulsas, o dataset traz pares com a versão vulnerável e a versão corrigida da mesma função, e propõe um protocolo de avaliação mais próximo do uso real de uma ferramenta de segurança.

Formato: seis arquivos JSON Lines: train, valid e test, cada um completo e pareado (`_paired`). Cada linha traz o código em **func**, o rótulo em **target** e campos de rastreo (projeto, commit).

2. Por que foi criado

Rótulos errados. Datasets anteriores marcavam como vulnerável toda função alterada por um commit de correção, inclusive alterações de consistência/formatação ou casos em que a falha estava espalhada por várias funções. Na revisão manual dos autores, 40% a 76% das funções consideradas vulneráveis nesses datasets não eram vulneráveis.

Duplicatas. A mesma função podia aparecer mais de uma vez, inclusive em treino e teste, às vezes com rótulos contraditórios.

Viagem no tempo. Divisões aleatórias permitem treinar com código mais novo que o teste e podem colocar funções do mesmo commit em conjuntos diferentes.

Métricas pouco realistas. Acurácia e F1 não mostram bem o equilíbrio entre vulnerabilidades perdidas e alarmes falsos.

3. Como foi construído

1. Juntar: reúne commits de segurança e funções alteradas por eles do BigVul, CrossVul, CVEfixes e DiverseVul. O Devign/CodeXGLUE fica de fora.

2. Deduplicar: normaliza caracteres de formatação, calcula o hash MD5 de cada função e remove cópias.

3. Rotular vulneráveis: uma função é marcada como vulnerável se passar por ONEFUNC ou NVDCHECK.

4. Rotular benignas: são a versão corrigida de cada função vulnerável e todas as funções não alteradas dos mesmos commits.

5. Dividir no tempo: 80% das amostras mais antigas para treino, 10% seguintes para validação e 10% mais recentes para teste. Funções do mesmo commit permanecem no mesmo conjunto.

ONEFUNC: a função é a única alterada pelo commit de correção de segurança. Revisão manual: 86% corretas.

NVDCHECK: a descrição do CVE no NVD cita o nome da função; ou cita o arquivo e ela é a única função alterada pelo commit nesse arquivo. Revisão manual: 92% corretas.

4. Tamanho e divisão

Conjunto	Funções	Vulneráveis	Benignas	% vulneráveis	Pares
Treino	184.427	5.574	178.853	3,0%	4.354
Validação	25.430	699	24.731	2,7%	562
Teste	25.911	695	25.216	2,7%	564
Total	235.768	6.968	228.800	3,0%	5.480

O desbalanceamento é proposital. No teste há 1 vulnerável para cada 36 benignas; um modelo que responde sempre “não” teria 97,3% de acurácia. Os pares juntam a versão vulnerável e a corrigida da mesma função, com pelo menos 80% do texto em comum.

5. Como avaliar um modelo

VD-S (Vulnerability Detection Score): taxa de falsos negativos quando o limiar é ajustado para no máximo 0,5% de falsos positivos ($\text{FNR @ FPR} \leq 0,5\%$). Menor é melhor. No teste, 0,5% das 25.216 benignas corresponde a no máximo 126 alarmes falsos.

Avaliação pareada: o modelo classifica as duas versões de cada par.

Resultado	Vulnerável	Corrigida	Significado
P-C	sim	não	par correto
P-V	sim	sim	diz vulnerável às duas
P-B	não	não	diz benigna às duas
P-R	não	sim	par invertido

A avaliação pareada mostra se o modelo enxerga a correção ou apenas o “jeito” do código. Chute aleatório: P-C $\approx$ 25%.

6. O que o artigo encontrou

Modelo	Parâmetros	F1 BigVul	F1 PrimeVul	Acurácia	VD-S	P-C
CodeT5	60 M	64,9%	19,7%	96,7%	89,9%	1,1%
CodeBERT	125 M	62,9%	20,9%	96,9%	88,8%	1,8%
UniXcoder	125 M	65,5%	21,4%	96,9%	89,2%	1,6%
StarCoder2	7 B	68,3%	18,1%	97,0%	89,6%	2,3%
CodeGen2.5	7 B	67,3%	19,6%	96,7%	91,5%	3,0%

Os benchmarks antigos inflavam os modelos. O StarCoder2 treinado no BigVul tem 68,3% de F1 no BigVul e cai para 3,1% quando testado no PrimeVul; mesmo treinado no PrimeVul, chega a 18,1%.

Acurácia fica perto de 97% em todos os modelos, enquanto o VD-S mostra que cerca de 90% das vulneráveis escapam dentro do limite de alarmes.

Na avaliação pareada, os modelos ajustados acertam de 1% a 3% dos pares. GPT-3.5 e GPT-4 ficam entre 1,2% e 12,9%, contra 22,7% de um chute aleatório.

7. Versões disponíveis

Original: release usada no artigo, com os números desta página. É a que permite comparar com a literatura.

v0.1 (set/2024): acrescenta metadados como CVE e CWE, mas mantém somente vulnerabilidades cujos metadados foram recuperados. É um subconjunto: teste com 24.788 funções, 549 vulneráveis e 435 pares. Os resultados das duas versões não são comparáveis.

8. Limitações

Só C/C++ e somente a função isolada: o modelo não vê quem chama a função, o restante do arquivo nem o commit.

Os rótulos são melhores, mas não perfeitos: 8% a 14% de erro nas vulneráveis revisadas.

Há poucas vulneráveis no teste (695) e poucos pares (564), então diferenças pequenas entre modelos podem ser acaso.

O limite de 0,5% do VD-S é uma escolha; outro valor muda o número.

Fonte: Ding et al. (ICSE 2025), Tabelas I, III, V e VIII; contagens da v0.1: README oficial do PrimeVul.